

# Parallel Computing Exercises

HRL Uni Bonn

April 29, 2026

## Assignment 1

Your first assignment is to gain some understanding of the CUDA programming framework using NVIDIA CUDA tutorials and the programming guide. Make sure you understand the concept of blocks and threads and the different sorts of memory you can use. We created repositories for you on our Gitlab server. The repository contains instructions on how to connect to our computers, set up the work with CUDA, and also links to the programming guide and documentation you can use.

To connect to our computers, you have to use your GSG / computer science account, but if you have a private computer with an NVIDIA GPU, you can also use that to start.

**Please write a program that:**

1. Initializes two vectors of 900000000 (900 million) doubles with random numbers.
2. Adds these two vectors on the GPU.
3. Adds these vectors on the CPU as well.
4. Compares the GPU and CPU results to check for errors.
5. Measures and prints the runtimes of each step such as the initialization of memory, copying of data between host and device, execution time of the kernel, and the execution time of adding the vectors on the CPU.
6. Measures and prints the bandwidth of the data flow between host and device.

Experiment with the parameters such as the number of blocks and the number of threads per block and observe the effect on the runtime. Also try out different ways of managing the memory, like managed memory versus manually allocated memory on the GPU. Note that you can't measure transfer speed with managed memory.

Aside from the addition, the initialization with random values is also a time consuming task. Consider a parallelized initialization on the GPU. You may also try both initializing on the CPU and GPU, and compare the runtime.

## Assignment 2

Your next assignment will be to find the maximum element in a vector of 900000000 floats. Still 900 million, but this time floats!

**Your program should...**

1. Allocate the memory needed to store 900 million numbers and initialize the memory with random numbers between 0 and 1. Hint: it is faster to initialize the vector on the GPU.
2. Find the index and the value of the largest number using the GPU and the CPU.
3. Output the index and the value found on the GPU and the CPU for comparison.
4. Print a comprehensive log to the command line stating the runtimes of different sections of your program.

## Assignment 3

The goal of this assignment is to: Allow you to explore GPU programming with CUDA and understand how CUDA can handle computationally expensive tasks like Chamfer Distance calculations (You may find the equation here: <https://github.com/UM-ARM-Lab/Chamfer-Distance-API>), and to encourage you to evaluate and compare the performance of GPU-based solutions against optimized CPU-based solutions (the KD-Tree for nearest-neighbor searches).

Therefore for this assignment, we provide you with startup code that includes:

- A CPU implementation of Chamfer Distance using brute force.
- A CPU implementation of Chamfer Distance using KD-Tree.
- Utilities to generate random point clouds in 3D space.
- A basic framework for testing performance.

Your task is to implement the CUDA version of brute-force Chamfer Distance using the provided startup code, by following these steps:

1. Write a CUDA kernel to compute the Chamfer Distance between two point clouds, by:
  - (a) Compute the nearest neighbor for each point in the first point cloud (cloud1) in the second point cloud (cloud2).
  - (b) Reverse the process (from cloud2 to cloud1).
  - (c) Combine the results to compute the Chamfer Distance.
2. Performance Comparison, comparing results and runtime of your CUDA implementation with:
  - (a) The CPU brute-force implementation.
  - (b) The CPU KD-Tree implementation.
3. Validate the following for various values of N (i.e. 100, 1000, 10000, 100000, 1000000, 2000000):
  - (a) Chamfer Distance values computed by the CUDA, CPU brute-force, and CPU KD-Tree methods.
  - (b) Execution times for all three implementations.
  - (c) Consistency check to verify the correctness of the CUDA results.

## Assignment 4

This final assignment is a simplified 3D inspection pipeline commonly needed in mobile robotics. In the context of Parallel Computing for Mobile Robots, we explore how a mobile robot could inspect objects by generating partial observations and aggregating them into a 3D model.

### Background

Model-based inspection is crucial for tasks like environment mapping, warehouse inventory monitoring, or object scanning in manufacturing. Such pipelines often rely on:

- A 3D model of the object or environment.
- A predefined set of viewpoints from which the robot can observe the object.
- Ray casting from these viewpoints to determine which parts (voxels) of the model are visible.
- Selection of the next best views (NBV) to maximize coverage of the model.

The proposed pipeline is closely related to other 3D inspection frameworks. In particular, it is a similar method like those described by Jing and Shimada (2018)<sup>1</sup> with a focus on a model-based scenario (illustrated by a drone inspecting a building), as opposed to online active exploration (illustrated by a mobile arm reconstructing an object) detailed in Lee et al. (2022)<sup>2</sup>.

These works are referenced solely to provide context for our model-based inspection pipeline and are not directly relevant to the coding in this assignment. In both references, the system identifies which parts of a 3D object are visible from certain viewpoints, then decides the subsequent viewpoints to achieve complete coverage. The simplified code provided here illustrates similar concepts: the object is discretized into a voxel grid, and simplified ray casting is performed from various viewpoints. Assuming the object is entirely within the camera's field of view, ray casting is confined to 3D space. The visible voxels are then computed, and the next best view is planned to maximize the discovery of newly observed regions.

### Task

We will give you a CPU-based implementation of:

- A voxel grid class (VoxelGrid) to represent objects and maintain occupancy data;

---

<sup>1</sup>Jing, Wei, and Kenji Shimada; Model-based view planning for building inspection and surveillance using voxel dilation, medial objects, and random-key genetic algorithm.

<sup>2</sup>Lee, Soomin, et al.; Uncertainty guided policy for active robotic 3d reconstruction using neural radiance fields.

- Ray casting logic that checks which voxels become visible from a given viewpoint;
- A Chamfer Distance function to measure the overlap or error between two-point clouds (e.g., a reconstruction vs. the full model).

However, the CPU version becomes very slow for larger point clouds and higher voxel resolutions. The primary goal of this assignment is to modify the code to run in a reasonable amount of time, ideally under 10 minutes even for larger voxel resolutions and multiple viewpoints.

Implement the following:

1. Implement CUDA kernels for ray casting in the voxel grid.
2. Implement CUDA kernels for chamfer distance brute-force search (Assignment03).

These CUDA kernels should significantly accelerate the computation compared to the pure CPU implementations.

Optional Optimizations:

1. Batch or compressed ray casting: Instead of casting individual rays serially for each viewpoint, bundle or compress them to better leverage GPU parallelism.
2. Parallel Hash Table in NBV selection: If time permits, consider parallelizing the computation of the next best view and optimizing the management of hash tables for identifying duplicate voxels using GPU-friendly data structures.

By offloading computationally expensive tasks to the GPU, we aim to handle the entire pipeline—from voxelization and ray casting to NBV selection and Chamfer Distance evaluation—within feasible time limits.

## Code Overview

Given that you have not previously been exposed to the complete system, below is a brief breakdown of the key code components:

1. VoxelGrid
  - (a) Data: 3D occupancy stored in a 1D vector (grid) of booleans.
  - (b) Methods:
    - insertPointCloud: Mark the corresponding voxels as occupied for each point.
    - rayCasting: Step through the grid along a ray direction until hitting an occupied voxel or reaching the max distance.
    - extractPointCloud: Convert occupied voxels back into a point cloud.

## 2. Ray Casting

- (a) A CPU-based traversal stepping in increments of  $\text{voxelSize} / 2.0$  along each ray.
- (b) Checking occupancy in the grid involves indexing into the boolean array.
- (c) For CUDA acceleration, you should implement this logic in a kernel that handles many rays in parallel, significantly speeding up the process.

## 3. Viewpoints & NBV (Next Best View)

- (a) The code reads 100 viewpoints from `Viewspace_100.txt`.
- (b) For each viewpoint, ray cast to find visible voxels.
- (c) The system keeps track of newly observed voxels to select the viewpoint that maximizes the number of visible voxels (the coverage of the object).
- (d) Hash tables or sets store unique voxel indices. Speeding up insertion/checks in a GPU context requires advanced data structures or parallel algorithms (optional).

## 4. Input & Output

- (a) PCD reading (`readPCD`) and writing (`savePCD`) to handle point cloud files.
- (b) Outputs:
  - Per-viewpoint visible voxels as `.pcd`.
  - Combined “all visible” or “merged NBV” `.pcd`.
  - Logging: Number of visible points, Chamfer distance, Runtime, etc.

Note: PCD is a standard file format for point cloud data. We provide a Python script, `VisualizePCD.py`, to help you visualize PCD files. To use it, ensure you have Open3D installed in your Python environment, then run the following command: `python VisualizePCD.py -input_file {your_pcd_file_name}.pcd`

The pipeline begins by loading voxelized models (e.g., `Armadillo_XXX.pcd`, `Dragon_XXX.pcd`) at different resolutions (64, 128, 256). For each model and resolution, a `VoxelGrid` is created, and `insertPointCloud` is called to mark occupied voxels. Next, the CPU loop over voxels and viewpoint rays is identified. This should be replaced with a CUDA kernel to parallelize the process of checking visibility. In the NBV selection step, your task includes implementing a CUDA kernel to compute the chamfer distance. If time permits, you may also choose to parallelize the NBV selection process.